

# WEST BENGAL STATE UNIVERSITY



## Department of Computer Science SOFTWARE ENGINEERING ASSIGNMENT

Name : Avijit Bose

Semester : 4<sup>th</sup> Sem/2<sup>nd</sup> Year

Paper Code : CMSPCOR15P

Roll Number : 10024016900015

Registration Number : 1032111400443

Submitted to : Dr. Rajat Pandit

Session : 2024–2026

# Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Software Cost Estimation Using the Basic COCOMO Model</b> | <b>1</b>  |
| 1.1      | Program to Estimate Cost using KLOC . . . . .                | 1         |
| 1.1.1    | Python Program . . . . .                                     | 1         |
| 1.1.2    | Output . . . . .                                             | 3         |
| 1.1.3    | Graph . . . . .                                              | 4         |
| 1.2      | Program to Estimate Cost using Function Point . . . . .      | 5         |
| 1.2.1    | Python Program . . . . .                                     | 5         |
| 1.2.2    | Sample Output . . . . .                                      | 7         |
| 1.2.3    | Graph . . . . .                                              | 8         |
| <b>2</b> | <b>Norden Model</b>                                          | <b>9</b>  |
| 2.0.1    | Python Program . . . . .                                     | 9         |
| 2.0.2    | Sample Output . . . . .                                      | 10        |
| 2.0.3    | Graph . . . . .                                              | 11        |
| <b>3</b> | <b>Putnam Model</b>                                          | <b>12</b> |
| 3.0.1    | Python Program . . . . .                                     | 12        |
| 3.0.2    | Sample Output . . . . .                                      | 13        |
| 3.0.3    | Graph . . . . .                                              | 14        |
| <b>4</b> | <b>Jensen Model</b>                                          | <b>15</b> |
| 4.0.1    | Python Program . . . . .                                     | 15        |
| 4.0.2    | Sample Output . . . . .                                      | 16        |
| 4.0.3    | Graph . . . . .                                              | 17        |
| <b>5</b> | <b>McCabe's Cyclomatic Complexity Metric</b>                 | <b>18</b> |
| 5.0.1    | Python Program . . . . .                                     | 18        |
| 5.0.2    | Sample Output . . . . .                                      | 20        |
| 5.0.3    | Control Flow Graph . . . . .                                 | 21        |
| <b>6</b> | <b>Halstead Software Metrics</b>                             | <b>22</b> |
| 6.0.1    | Input Program (halsted_input.py) . . . . .                   | 22        |
| 6.0.2    | Python Program . . . . .                                     | 22        |
| 6.0.3    | Sample Output . . . . .                                      | 24        |
| 6.0.4    | Graph . . . . .                                              | 25        |
| <b>7</b> | <b>Unit Testing</b>                                          | <b>26</b> |
| 7.0.1    | Python Program . . . . .                                     | 26        |

---

|       |                                          |    |
|-------|------------------------------------------|----|
| 7.0.2 | Sample Output . . . . .                  | 27 |
| 7.1   | Path Testing . . . . .                   | 28 |
| 7.1.1 | Python Program . . . . .                 | 28 |
| 7.1.2 | Sample Output . . . . .                  | 32 |
| 7.1.3 | Control Flow Graph . . . . .             | 33 |
| 7.2   | Boundary Value Analysis . . . . .        | 34 |
| 7.2.1 | Python Program . . . . .                 | 34 |
| 7.2.2 | Sample Output . . . . .                  | 37 |
| 7.3   | Equivalence Class Partitioning . . . . . | 39 |
| 7.3.1 | Python Program . . . . .                 | 39 |
| 7.3.2 | Sample Output . . . . .                  | 42 |
| 7.4   | Data Flow Testing . . . . .              | 44 |
| 7.4.1 | Python Program . . . . .                 | 44 |
| 7.4.2 | Sample Output . . . . .                  | 48 |
| 7.4.3 | Control Flow Graph . . . . .             | 49 |

# 1 Software Cost Estimation Using the Basic CO-COMO Model

## 1.1 Program to Estimate Cost using KLOC

### Objective

To estimate the software development effort, development time, team size and total project cost using the Basic COCOMO Model based on the software size measured in Kilo Lines of Code (KLOC).

#### 1.1.1 Python Program

```
1  import matplotlib.pyplot as plt
2  # COCOMO coefficient matrix [a, b, c, d]
3  kloc_matrix = [
4      [2.4, 1.05, 2.5, 0.38],    # Organic
5      [3.0, 1.12, 2.5, 0.35],    # Semi-detached
6      [3.6, 1.20, 2.5, 0.32]    # Embedded
7  ]
8  mode_names = ["Organic", "Semi-Detached", "Embedded"]
9  colors = ["blue", "green", "red"]
10 kloc = float(input("Enter KLOC (Kilo Lines of Code): "))
11 cost_per_pm = float(input("Enter cost per person-month: "))
12 print("\n--- COCOMO Estimation ---")
13 # Calculate output for each mode
14 for i in range(3):
15     a, b, c, d = kloc_matrix[i]
16     effort = a * (kloc ** b)
17     development_time = c * (effort ** d)
18     team_size = effort / development_time
19     total_cost = effort * cost_per_pm
20
21     print("\nMode:", mode_names[i])
22     print("Estimated Effort(Person-Months):", round(effort, 2))
23     print("Development Time(Months):", round(development_time, 2))
24     print("Team size (Persons):", round(team_size, 2))
25     print("Total estimated cost:", round(total_cost, 2))
26 # Graph data
27 kloc_values = list(range(1, 51))
```

```
28 plt.figure()
29 for i in range(3):
30     a, b, c, d = kloc_matrix[i]
31     effort_values = []
32     dev_time_values = []
33     team_size_values = []
34     cost_values = []
35     for k in kloc_values:
36         e = a * (k ** b)
37         t = c * (e ** d)
38         p = e / t
39         cost = e * cost_per_pm
40         effort_values.append(e)
41         dev_time_values.append(t)
42         team_size_values.append(p)
43         cost_values.append(cost)
44
45     plt.subplot(2,2,1)
46     plt.plot(kloc_values, effort_values, color=colors[i],
47             ↪ label=mode_names[i])
48     plt.xlabel("KLOC")
49     plt.ylabel("Effort")
50     plt.title("KLOC vs Effort")
51     plt.grid(True)
52     plt.subplot(2,2,2)
53     plt.plot(kloc_values, dev_time_values, color=colors[i],
54             ↪ label=mode_names[i])
55     plt.xlabel("KLOC")
56     plt.ylabel("Development Time")
57     plt.title("KLOC vs Development Time")
58     plt.grid(True)
59     plt.subplot(2,2,3)
60     plt.plot(kloc_values, team_size_values, color=colors[i],
61             ↪ label=mode_names[i])
62     plt.xlabel("KLOC")
63     plt.ylabel("Team Size")
64     plt.title("KLOC vs Team Size")
65     plt.grid(True)
66     plt.subplot(2,2,4)
67     plt.plot(kloc_values, cost_values, color=colors[i], label=mode_names[i])
```

```
65     plt.xlabel("KLOC")
66     plt.ylabel("Total Cost")
67     plt.title("KLOC vs Total Cost")
68     plt.grid(True)
69
70 for i in range(1,5):
71     plt.subplot(2,2,i)
72     plt.legend()
73
74 plt.tight_layout()
75 plt.show()
```

### 1.1.2 Output

```
(.venv) PS C:\SE> & c:\SE\.venv\Scripts\python.exe c:/SE/cocomo.py
Enter KLOC (Kilo Lines of Code): 25
Enter cost per person-month: 50000
--- COCOMO Estimation ---
Mode: Organic
Estimated Effort(Person-Months): 70.48
Development Time(Months): 12.6
Team size (Persons): 5.6
Total estimated cost: 3523856.83
Mode: Semi-Detached
Estimated Effort(Person-Months): 110.36
Development Time(Months): 12.97
Team size (Persons): 8.51
Total estimated cost: 5518030.15
Mode: Embedded
Estimated Effort(Person-Months): 171.33
Development Time(Months): 12.96
Team size (Persons): 13.21
Total estimated cost: 8566442.72
```

### 1.1.3 Graph

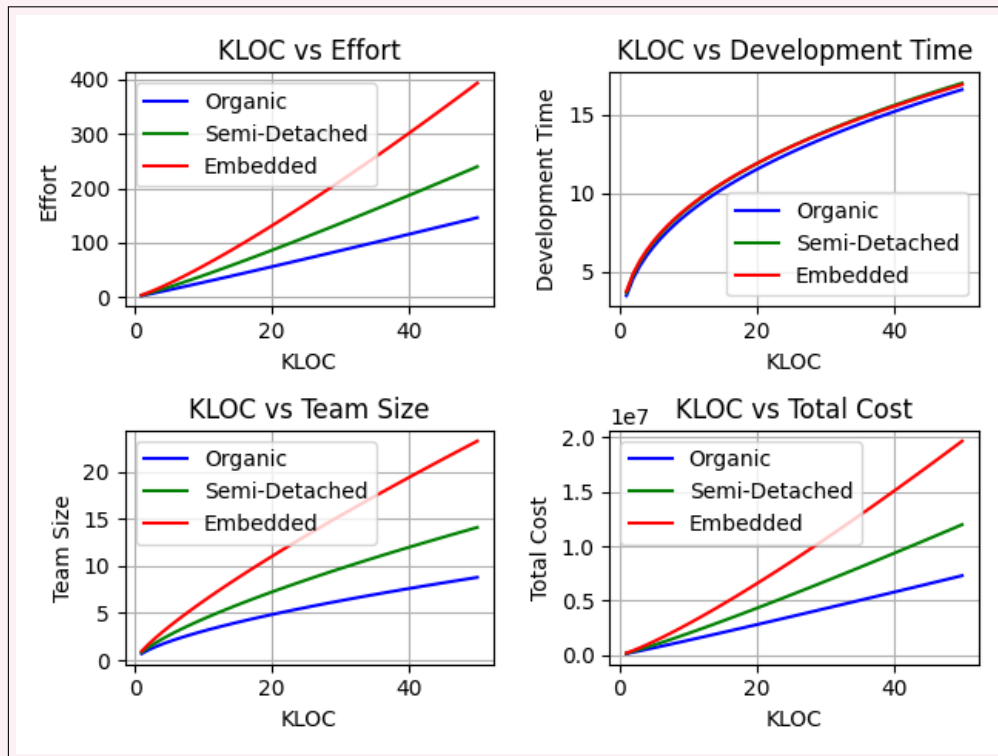


Figure 1: Graphs showing KLOC vs Effort, Development Time, Team Size and Total Cost

### Result

The program successfully estimated the effort, development time, team size and total software development cost for Organic, Semi-Detached and Embedded software projects using the Basic COCOMO model. The generated graphs show that as the software size increases, the effort, development time and project cost also increase.

## 1.2 Program to Estimate Cost using Function Point

### Objective

To estimate the software development effort, development time, staffing requirement and total project cost using Function Point Analysis and the Basic COCOMO Model.

#### 1.2.1 Python Program

```
1  import matplotlib.pyplot as plt
2  # Function Point weight matrix
3  # [EI, EO, EQ, ILF, EIF]
4  fp_matrix = [
5      [3, 4, 3, 7, 5], # Low complexity
6      [4, 5, 4, 10, 7], # Average complexity
7      [6, 7, 6, 15, 10] # High complexity
8  ]
9  complexity_names = ["Low", "Average", "High"]
10 colors = ["blue", "green", "red"]
11 # Input counts
12 EI = int(input("Enter number of External Inputs (EI): "))
13 EO = int(input("Enter number of External Outputs (EO): "))
14 EQ = int(input("Enter number of External Inquiries (EQ): "))
15 ILF = int(input("Enter number of Internal Logical Files (ILF): "))
16 EIF = int(input("Enter number of External Interface Files (EIF): "))
17 # Value Adjustment Factor
18 total_fi = int(input("Enter sum of 14 General System Characteristics (0-70):
19 ↪ "))
20 VAF = 0.65 + (0.01 * total_fi)
21 # FP to KLOC conversion factor
22 fp_to_kloc = float(input("Enter FP to KLOC conversion factor: "))
23 cost_per_pm = float(input("Enter cost per person-month: "))
24 print("\n--- Function Point Estimation ---")
25 # Graph data container
26 kloc_values = []
27 effort_values = []
28 dev_time_values = []
29 team_size_values = []
30 cost_values = []
31 for i in range(3):
32     w_EI, w_EO, w_EQ, w_ILF, w_EIF = fp_matrix[i]
```



```

32     # UFP
33     UFP = (EI * w_EI) + (EO * w_EO) + (EQ * w_EQ) + (ILF * w_ILF) + (EIF *
    ↪ w{EIF)
34     # Adjusted FP
35     AFP = UFP * VAF
36     # Convert to KLOC
37     kloc = AFP * fp_to_kloc
38     # Basic COCOMO (Organic assumption)
39     a, b, c, d = 2.4, 1.05, 2.5, 0.38
40     effort = a * (kloc ** b)
41     dev_time = c * (effort ** d)
42     team_size = effort / dev_time
43     total_cost = effort * cost_per_pm
44     print("\nComplexity:", complexity_names[i])
45     print("UFP:", round(UFP,2))
46     print("Adjusted FP:", round(AFP,2))
47     print("KLOC:", round(kloc,2))
48     print("Effort(Person-Months):", round(effort,2))
49     print("Development Time(Months):", round(dev_time,2))
50     print("Team Size:", round(team_size,2))
51     print("Total Cost:", round(total_cost,2))
52     kloc_values.append(kloc)
53     effort_values.append(effort)
54     dev_time_values.append(dev_time)
55     team_size_values.append(team_size)
56     cost_values.append(total_cost)
57     # Graphs
58     plt.figure()
59     plt.subplot(2,2,1)
60     plt.bar(complexity_names, effort_values, color=colors)
61     plt.title("Effort by Complexity")
62     plt.ylabel("Effort")
63     plt.subplot(2,2,2)
64     plt.bar(complexity_names, dev_time_values, color=colors)
65     plt.title("Development Time")
66     plt.subplot(2,2,3)
67     plt.bar(complexity_names, team_size_values, color=colors)
68     plt.title("Team Size")
69     plt.subplot(2,2,4)
70     plt.bar(complexity_names, cost_values, color=colors)

```

```
71 plt.title("Total Cost")
72 plt.tight_layout()
73 plt.show()
```

### 1.2.2 Sample Output

```
PS C:\SE> & c:\SE\.venv\Scripts\python.exe "c:/SE/cost_estimation(FP).py"
Enter number of External Inputs (EI): 20
Enter number of External Outputs (EO): 15
Enter number of External Inquiries (EQ): 10
Enter number of Internal Logical Files (ILF): 8
Enter number of External Interface Files (EIF): 5
Enter sum of 14 General System Characteristics (0-70): 10
Enter FP to KLOC conversion factor: 0.001
Enter cost per person-month: 50000

--- Function Point Estimation ---

Complexity: Low
UFP: 231
Adjusted FP: 173.25
KLOC: 0.17
Effort(Person-Months): 0.38
Development Time(Months): 1.73
Team Size: 0.22
Total Cost: 19045.31

Complexity: Average
UFP: 310
Adjusted FP: 232.5
KLOC: 0.23
Effort(Person-Months): 0.52
Development Time(Months): 1.95
Team Size: 0.27
Total Cost: 25937.33

Complexity: High
UFP: 455
Adjusted FP: 341.25
```

KLOC: 0.34  
Effort(Person-Months): 0.78  
Development Time(Months): 2.27  
Team Size: 0.34  
Total Cost: 38806.77

### 1.2.3 Graph

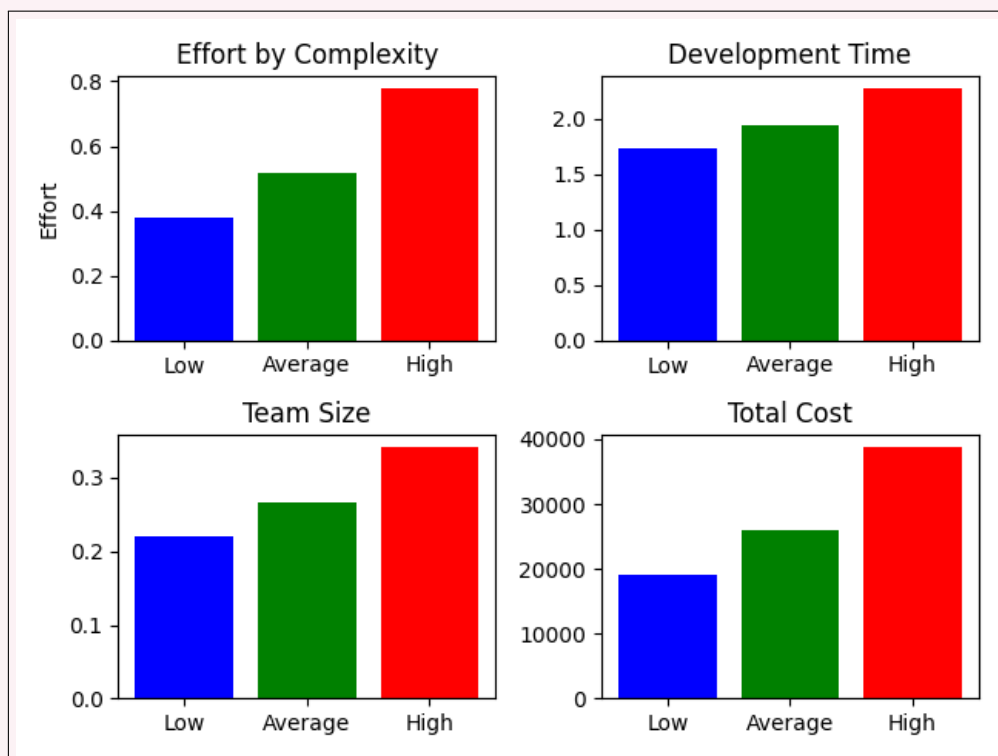


Figure 2: Comparison of Effort, Development Time, Team Size and Total Cost for Different Function Point Complexity Levels

### Result

The Function Point Analysis method successfully estimated the software size based on functional requirements. After converting Function Points into KLOC, the Basic CO-COMO model was applied to estimate the software development effort, development time, staffing requirement and overall project cost. This approach is useful during the early stages of software development when the exact number of lines of code is not yet known.

## 2 Norden Model

### Objective

To estimate the distribution of software development effort over time using Norden's Rayleigh Curve Model and visualize the effort distribution graphically.

#### 2.0.1 Python Program

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  # Inputs
4  K = float(input("Enter Total Effort K (person-years, e.g. 10): "))
5  td = float(input("Enter Peak Time t_d (years, e.g. 2): "))
6  # Time values
7  t = np.linspace(0.01, 5 * td, 200)
8  # Norden's Rayleigh formula
9  E = (K / td**2) * t * np.exp(-t**2 / (2 * td**2))
10 print("\n---Norden's Model Output---")
11 print(f"Total Effort (K): {K}")
12 print(f"Peak Time (td): {td}")
13 # Peak effort formula: (K/td) * e-0.5
14 E_peak = (K / td) * np.exp(-0.5)
15 print(f"Peak Effort: {E_peak:.4f}")
16 # Create figure and axis
17 fig, ax = plt.subplots()
18 # Plot curve
19 ax.plot(t, E, color='black', linewidth=2)
20 # Axis labels
21 ax.set_xlabel("Time")
22 ax.set_ylabel("Effort per unit time")
23
24 fig.suptitle("Rayleigh Curve | Norden's Model", fontsize=16, y=0.9, x=0.55)
25 ax.set_title(
26     r"$E(t) = \dfrac{K}{t_d^2} \times t \times e^{-t^2/(2t_d^2)}$",
27     fontsize=13,
28     pad=5
29 )
30 ax.axvline(td, color='red', linestyle='--', linewidth=1.5)
31 ax.text(td+0.2, max(E)*0.05, "td", color='red')
```

```
32 ax.text(max(t)*0.65, max(E)*0.8,  
33         f"K = {K}\ntd = {td}",  
34         bbox=dict(facecolor='white', edgecolor='black'))  
35  
36 ax.grid(alpha=0.3)  
37 plt.tight_layout(rect=[0, 0, 1, 0.92])  
38 plt.savefig("norden.png")  
39 plt.show()
```

## 2.0.2 Sample Output

```
Enter Total Effort K (person-years, e.g. 10): 10  
Enter Peak Time t_d (years, e.g. 2): 2  
  
--- Norden's Model Output ---  
Total Effort (K): 10.0  
Peak Time (td): 2.0  
Peak Effort: 3.0327
```

### 2.0.3 Graph

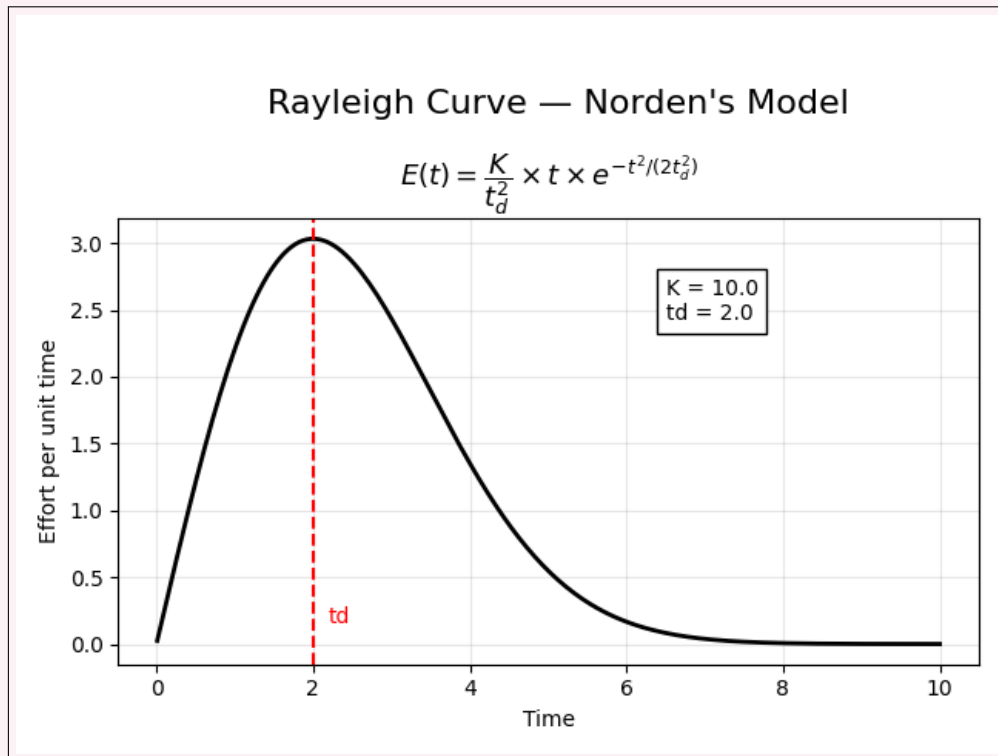


Figure 3: Rayleigh Curve representing Norden's Software Development Model

### Result

The Norden Model successfully estimated the effort distribution throughout the software development process using the Rayleigh curve. The generated graph shows that the development effort gradually increases, reaches a maximum at the peak development time ( $t_d$ ), and then decreases as the project approaches completion. This model is useful for estimating manpower requirements and planning software projects effectively.

## 3 Putnam Model

### Objective

To estimate the software development effort, average staffing requirement, and project cost using the Putnam Software Cost Estimation Model and visualize the relationship between development time and effort.

#### 3.0.1 Python Program

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  print("PUTNAM SOFTWARE COST ESTIMATION")
4  try:
5      # Input
6      loc = float(input("Enter software size (LOC): "))
7      ck = float(input("Enter technology constant: "))
8      time = float(input("Enter development time (years): "))
9      cost_per_year = float(input("Enter cost per person-year (Rs.): "))
10     # Calculations
11     effort = (loc / (ck * (time ** (4 / 3)))) ** 3
12     staff = effort / time
13     cost = effort * cost_per_year
14     # Display Result
15     print("\n----- Result -----")
16     print("Software Size      :", loc, "LOC")
17     print("Technology Constant:", ck)
18     print("Development Time    :", time, "years")
19     print("Effort                :", round(effort, 2), "person-years")
20     print("Average Staff        :", round(staff, 2), "persons")
21     print("Estimated Cost       : Rs.", round(cost, 2))
22     # Graph
23     x = np.linspace(time * 0.5, time * 3, 100)
24     y = (loc / (ck * (x ** (4 / 3)))) ** 3
25
26     plt.plot(x, y)
27     plt.scatter(time, effort)
28     plt.title("Putnam Model")
29     plt.xlabel("Development Time (Years)")
30     plt.ylabel("Effort (Person-Years)")
```

```
31     plt.grid(True)
32     plt.show()
33 except ValueError:
34     print("Please enter valid numeric values.")
```

### 3.0.2 Sample Output

```
PUTNAM SOFTWARE COST ESTIMATION

Enter software size (LOC): 50000
Enter technology constant: 8000
Enter development time (years): 2
Enter cost per person-year (Rs.): 800000

----- Result -----
Software Size      : 50000.0 LOC
Technology Constant: 8000.0
Development Time   : 2.0 years
Effort             : 15.26 person-years
Average Staff      : 7.63 persons
Estimated Cost     : Rs. 12207031.25
```



### 3.0.3 Graph

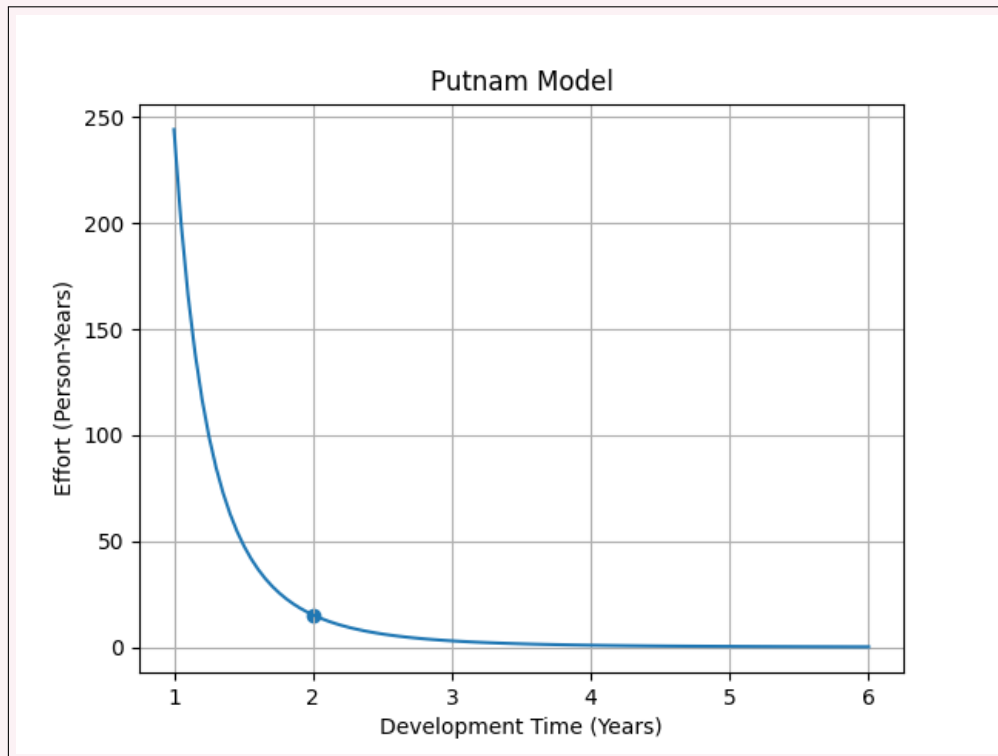


Figure 4: Effort Estimation using the Putnam Software Cost Estimation Model

### Result

The Putnam Model successfully estimated the software development effort, average staffing requirement, and overall project cost based on the software size, technology constant, and development time. The generated graph illustrates that as the development time changes, the required development effort also changes significantly. The model is useful for project planning and resource allocation during software development.

## 4 Jensen Model

### Objective

To estimate the software development effort, average staffing requirement, and project cost using the Jensen Software Cost Estimation Model and visualize the relationship between development time and development effort.

#### 4.0.1 Python Program

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3  print("JENSEN SOFTWARE COST ESTIMATION")
4  try:
5      # Input
6      loc = float(input("Enter software size (LOC): "))
7      ctc = float(input("Enter technology constant: "))
8      time = float(input("Enter development time (years): "))
9      cost_per_year = float(input("Enter cost per person-year (Rs.): "))
10     # Calculations
11     effort = (loc / (ctc * time)) ** 2
12     staff = effort / time
13     cost = effort * cost_per_year
14     # Output
15     print("\n----- Result -----")
16     print("Software Size      :", loc, "LOC")
17     print("Technology Constant:", ctc)
18     print("Development Time    :", time, "years")
19     print("Effort                :", round(effort, 2), "person-years")
20     print("Average Staff         :", round(staff, 2), "persons")
21     print("Estimated Cost       : Rs.", round(cost, 2))
22     # Graph
23     x = np.linspace(time * 0.5, time * 3, 100)
24     y = (loc / (ctc * x)) ** 2
25
26     plt.plot(x, y)
27     plt.scatter(time, effort)
28     plt.title("Jensen Model")
29     plt.xlabel("Development Time (Years)")
30     plt.ylabel("Effort (Person-Years)")
```

```
31     plt.grid(True)
32     plt.show()
33 except ValueError:
34     print("Please enter valid numeric values.")
```

#### 4.0.2 Sample Output

```
JENSEN SOFTWARE COST ESTIMATION

Enter software size (LOC): 50000
Enter technology constant: 8000
Enter development time (years): 2
Enter cost per person-year (Rs.): 800000

----- Result -----
Software Size      : 50000.0 LOC
Technology Constant: 8000.0
Development Time   : 2.0 years
Effort             : 9.77 person-years
Average Staff      : 4.88 persons
Estimated Cost     : Rs. 7812500.0
```

### 4.0.3 Graph

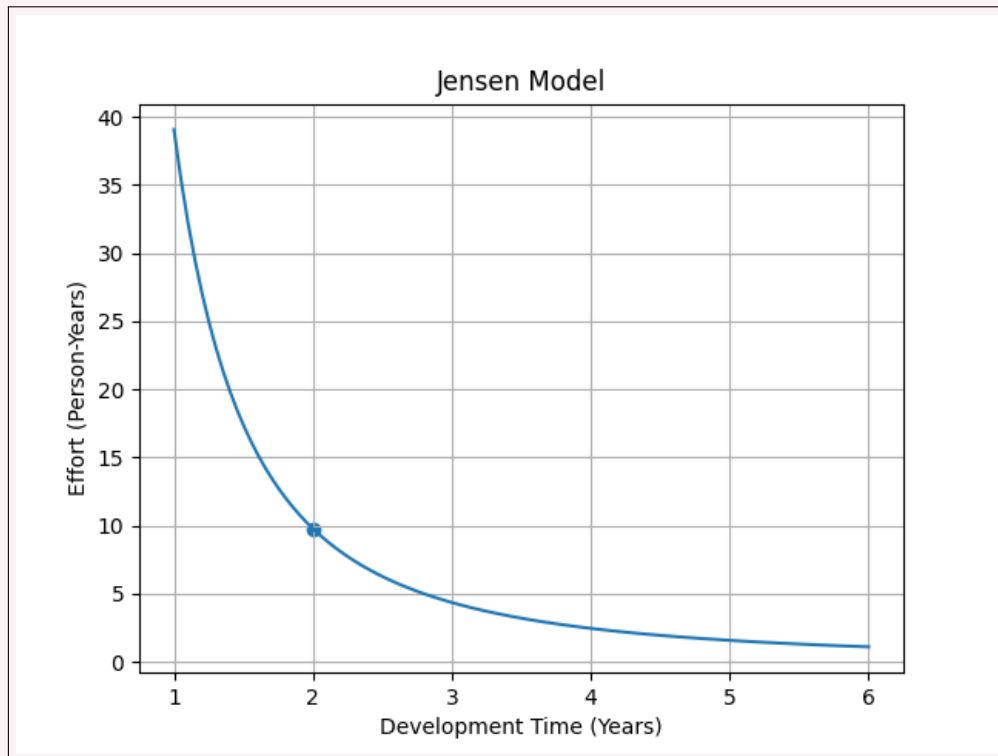


Figure 5: Effort Estimation using the Jensen Software Cost Estimation Model

### Result

The Jensen Model successfully estimated the software development effort, average staffing requirement, and project cost based on the software size, technology constant, and development time. The graph illustrates the relationship between development time and development effort, showing that a longer development time generally reduces the effort required. This model provides a simple and effective approach for software cost estimation during project planning.

## 5 McCabe's Cyclomatic Complexity Metric

### Objective

To calculate the Cyclomatic Complexity of a program using its Control Flow Graph (CFG), identify decision nodes, determine the risk level, and visualize the control flow graph.

#### 5.0.1 Python Program

```
1  import networkx as nx
2  import matplotlib.pyplot as plt
3  print("McCabe Cyclomatic Complexity")
4  choice = input("1. Enter your own graph\n2. Use sample graph\nEnter choice:
   ↪ ")
5  edges = []
6  if choice == "1":
7      print("Enter edges like 1,2")
8      print("Type 'done' to finish")
9      while True:
10         edge = input("Edge: ")
11         if edge.lower() == "done":
12             break
13         try:
14             a, b = edge.split(",")
15             edges.append((a.strip(), b.strip()))
16         except:
17             print("Invalid input")
18  else:
19      edges = [
20          ("1", "2"),
21          ("2", "3"),
22          ("2", "6"),
23          ("3", "4"),
24          ("3", "5"),
25          ("4", "6"),
26          ("5", "6"),
27          ("6", "2"),
28          ("6", "7")
29  ]
```

```
30     print("\nUsing sample graph")
31 if len(edges) == 0:
32     print("No edges entered.")
33     exit()
34 G = nx.DiGraph()
35 G.add_edges_from(edges)
36 E = G.number_of_edges()
37 N = G.number_of_nodes()
38 complexity = E - N + 2
39 decision_nodes = []
40 for node in G.nodes():
41     if G.out_degree(node) >= 2:
42         decision_nodes.append(node)
43 complexity2 = len(decision_nodes) + 1
44 print("\n----- Result -----")
45 print("Edges           :", E)
46 print("Nodes           :", N)
47 print("Decision Nodes    :", decision_nodes)
48 print("Cyclomatic (E-N+2) :", complexity)
49 print("Cyclomatic (P+1)   :", complexity2)
50 if complexity <= 10:
51     print("Risk Level       : Low")
52 elif complexity <= 20:
53     print("Risk Level       : Moderate")
54 elif complexity <= 50:
55     print("Risk Level       : High")
56 else:
57     print("Risk Level       : Very High")
58
59 colors = []
60 for node in G.nodes():
61     if node in decision_nodes:
62         colors.append("orange")
63     else:
64         colors.append("lightblue")
65 pos = nx.spring_layout(G)
66 nx.draw(
67     G,
68     pos,
69     with_labels=True,
```

```
70     node_color=colors,  
71     node_size=1000,  
72     arrows=True  
73 )  
74 plt.title("Control Flow Graph")  
75 plt.show()
```

### 5.0.2 Sample Output

```
McCabe Cyclomatic Complexity  
1. Enter your own graph  
2. Use sample graph  
Enter choice: 2  
  
Using sample graph  
  
----- Result -----  
Edges           : 9  
Nodes           : 7  
Decision Nodes  : ['2', '3', '6']  
Cyclomatic (E-N+2) : 4  
Cyclomatic (P+1)  : 4  
Risk Level      : Low
```

### 5.0.3 Control Flow Graph

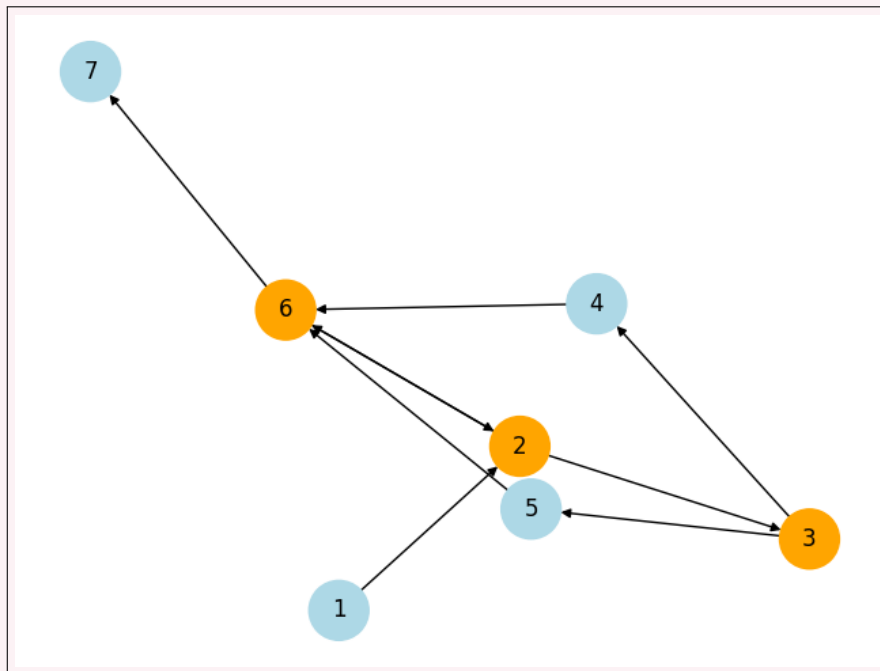


Figure 6: Control Flow Graph used for McCabe's Cyclomatic Complexity Analysis

#### Result

The program successfully calculated the Cyclomatic Complexity of the Control Flow Graph using both the  $E - N + 2$  and  $D + 1$  methods. In this example, the complexity value is 4, indicating four independent execution paths. Since the complexity is less than or equal to 10, the program is classified as having a **Low Risk** level and is considered relatively easy to understand, maintain, and test.



## 6 Halstead Software Metrics

### Objective

To analyze a Python program using Halstead Software Metrics by identifying operators and operands, computing the software complexity measures, and visualizing the operator and operand distribution.

#### 6.0.1 Input Program (halsted\_input.py)

```
1 x = 10
2 y = 5
3 z = x - y
4 if z > 10:
5     print("Result is greater than 10")
6 else:
7     print("Result is less than or equal to 10")
```

#### 6.0.2 Python Program

```
1 import tokenize
2 import math
3 import keyword
4 import matplotlib.pyplot as plt
5 def count_operators_and_operands(file_path):
6     operators = []
7     operands = []
8     with open(file_path, 'rb') as file:
9         tokens = tokenize.tokenize(file.readline)
10        for token in tokens:
11            # Operators
12            if token.type == tokenize.OP:
13                operators.append(token.string)
14            # Keywords treated as operators
15            elif token.type == tokenize.NAME and
16                ↪ keyword.iskeyword(token.string):
17                operators.append(token.string)
18            # Identifiers (variables, function names)
19            elif token.type == tokenize.NAME:
20                operands.append(token.string)
```

```

20         # Numbers & Strings → operands
21         elif token.type in (tokenize.NUMBER, tokenize.STRING):
22             operands.append(token.string)
23     return operators, operands
24 def halstead_metrics(N1, N2, n1, n2):
25     N = N1 + N2
26     n = n1 + n2
27     # Avoid log(0)
28     if n1 > 0 and n2 > 0:
29         N_hat = n1 * math.log2(n1) + n2 * math.log2(n2)
30     else:
31         N_hat = 0
32     V = N * math.log2(n) if n > 0 else 0
33     D = (n1 / 2) * (N2 / n2) if n2 > 0 else 0
34     E = D * V
35     print("\n--- Halstead Metrics ---")
36     print(f"Program Length (N): {N}")
37     print(f"Estimated Length (N): {N_hat:.2f}")
38     print(f"Vocabulary (n): {n}")
39     print(f"Volume (V): {V:.2f}")
40     print(f"Difficulty (D): {D:.2f}")
41     print(f"Effort (E): {E:.2f}")
42     return {
43         "N1": N1, "N2": N2,
44         "n1": n1, "n2": n2,
45         "N": N, "n": n,
46         "V": V, "D": D, "E": E
47     }
48 def plot_graphs(data):
49     labels = ['Operators (N1)', 'Operands (N2)']
50     values = [data["N1"], data["N2"]]
51     plt.figure(figsize=(20, 10))
52     # Pie chart
53     plt.subplot(1, 2, 1)
54     plt.pie(values, labels=labels, autopct='%1.1f%%', colors=['blue',
55         ↪ 'red'])
56     plt.title("Distribution")
57     # Bar chart
58     plt.subplot(1, 2, 2)
59     plt.bar(labels, values, color=['blue', 'red'])

```

```

59     plt.title("Operators vs Operands")
60     plt.tight_layout()
61     plt.show()
62 def main():
63     file_path = input("Enter the path to the Python file to analyze: ") #
        ↪ Path to the Python file to analyze
64     operators, operands = count_operators_and_operands(file_path)
65     ope=["++","--",":"]
66     operators.extend(ope)
67     print("\nOperands:", operands)
68     print("Operators:", operators)
69     N1 = len(operators)
70     N2 = len(operands)
71     n1 = len(set(operators))
72     n2 = len(set(operands))
73     data = halstead_metrics(N1, N2, n1, n2)
74     # Show graphs
75     plot_graphs(data)
76 if __name__ == '__main__':
77     main()

```

### 6.0.3 Sample Output

Enter the path to the Python file to analyze: halsted\_input.py

Operands:

```
['x', '10', 'y', '5', 'z', 'x', 'y', 'z', '10',
'print', '"Result is greater than 10"',
'print', '"Result is less than or equal to 10"']
```

Operators:

```
['=', '=', '=', '-', 'if', '>', ':',
'(', ')', 'else', ':', '(', ')',
'++', '--', ':']
```

--- Halstead Metrics ---

Program Length (N): 29

Estimated Length (N): 62.05

Vocabulary (n): 19

Volume (V): 123.19  
Difficulty (D): 8.94  
Effort (E): 1101.01

#### 6.0.4 Graph

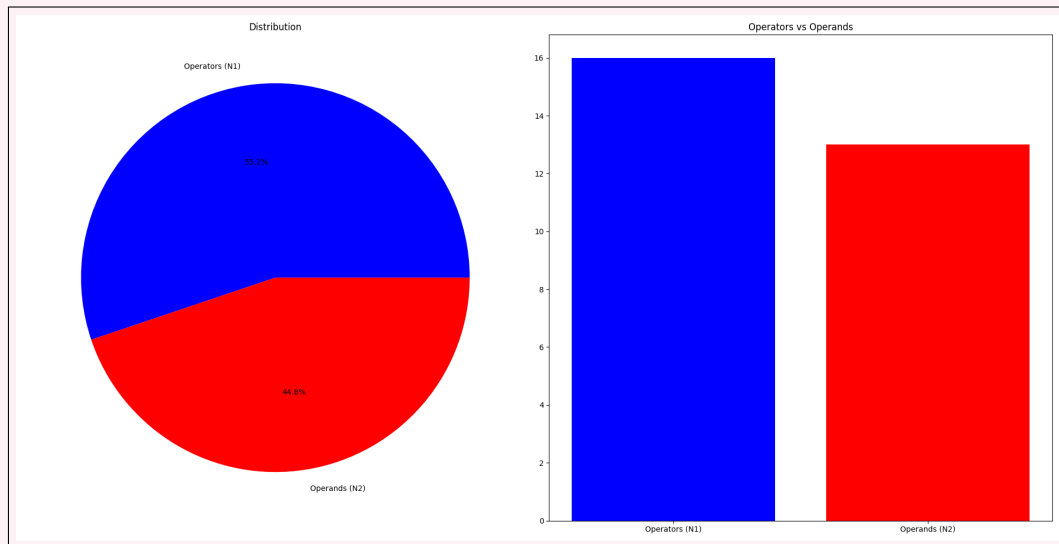


Figure 7: Distribution of Operators and Operands using Halstead Software Metrics

#### Result

The program successfully analyzed the given Python source code by identifying operators and operands and computing the Halstead Software Metrics. The calculated values of Program Length, Vocabulary, Volume, Difficulty, and Effort provide a quantitative measure of the software's complexity. The accompanying pie chart and bar graph visually represent the distribution of operators and operands, making the complexity analysis easier to interpret.

## 7 Unit Testing

### Objective

To verify the correctness of individual software components by testing each function independently and ensuring that every unit produces the expected output.

#### 7.0.1 Python Program

```
1  # Functions to be tested
2  def add(a, b):
3      return a + b
4  def subtract(a, b):
5      return a - b
6  def multiply(a, b):
7      return a * b
8  def divide(a, b):
9      if b == 0:
10         return "Division by zero is not allowed."
11     return a / b
12 # Test Cases
13 print("Unit Testing Results")
14 print("-" * 30)
15 # Test 1
16 expected = 15
17 actual = add(10, 5)
18 if expected == actual:
19     print("Test 1 (Addition): PASS")
20 else:
21     print("Test 1 (Addition): FAIL")
22 # Test 2
23 expected = 5
24 actual = subtract(10, 5)
25 if expected == actual:
26     print("Test 2 (Subtraction): PASS")
27 else:
28     print("Test 2 (Subtraction): FAIL")
29 # Test 3
30 expected = 50
31 actual = multiply(10, 5)
```

```
32 if expected == actual:
33     print("Test 3 (Multiplication): PASS")
34 else:
35     print("Test 3 (Multiplication): FAIL")
36 # Test 4
37 expected = 2
38 actual = divide(10, 5)
39 if expected == actual:
40     print("Test 4 (Division): PASS")
41 else:
42     print("Test 4 (Division): FAIL")
43 # Test 5
44 expected = "Division by zero is not allowed."
45 actual = divide(10, 0)
46 if expected == actual:
47     print("Test 5 (Division by Zero): PASS")
48 else:
49     print("Test 5 (Division by Zero): FAIL")
```

### 7.0.2 Sample Output

```
Unit Testing Results
-----
Test 1 (Addition): PASS
Test 2 (Subtraction): PASS
Test 3 (Multiplication): PASS
Test 4 (Division): PASS
Test 5 (Division by Zero): PASS
```

## Result

The unit testing program successfully verified the correctness of all arithmetic functions. Each test case produced the expected output, including the division-by-zero condition. Since all five test cases passed successfully, the implemented functions are considered to be functioning correctly.

## 7.1 Path Testing

### Objective

To identify all independent execution paths of a program using its Control Flow Graph (CFG), calculate the Cyclomatic Complexity, execute the program, and highlight the executed path.

### Input Program (input.py)

```
1  b = int(input("Enter a number: "))
2  if b > 0:
3      print("Positive")
4      EXECUTED_PATH.append("Positive")
5  elif b < 0:
6      print("Negative")
7      EXECUTED_PATH.append("Negative")
8  else:
9      print("Zero")
10     EXECUTED_PATH.append("Zero")
11 print("Program Finished")
```

#### 7.1.1 Python Program

```
1  import ast
2  import networkx as nx
3  import matplotlib.pyplot as plt
4  with open("path_testing/input.py", "r") as f:
5      code = f.read()
6  tree = ast.parse(code)
7  decision_count = 0
8  conditions = []
9  def extract_conditions(node):
10     global decision_count
11     for child in ast.iter_child_nodes(node):
12         if isinstance(child, ast.If):
13             decision_count += 1
14             conditions.append(ast.unparse(child.test))
15             extract_conditions(child)
16  extract_conditions(tree)
```

```
17 cyclomatic_complexity = decision_count + 1
18 paths = []
19 if decision_count == 2:
20     c1 = conditions[0]
21     c2 = conditions[1]
22     paths.append(
23         f"Start -> IF({c1}) TRUE -> End"
24     )
25     paths.append(
26         f"Start -> IF({c1}) FALSE -> IF({c2}) TRUE -> End"
27     )
28     paths.append(
29         f"Start -> IF({c1}) FALSE -> IF({c2}) FALSE -> End"
30     )
31 print("PATH TESTING ANALYSIS :-")
32 print(f"\nDecision Nodes = {decision_count}")
33 print(
34     f"Cyclomatic Complexity = {cyclomatic_complexity}"
35 )
36 print("\nIndependent Paths:\n")
37 for i, p in enumerate(paths, start=1):
38     print(f"Path {i}")
39     print(p)
40     print()
41 print("\nExecuting Program...\n")
42 EXECUTED_PATH = []
43 exec_globals = {
44     "EXECUTED_PATH": EXECUTED_PATH
45 }
46 exec(code, exec_globals)
47 print("\nExecuted Branch:")
48 if EXECUTED_PATH:
49     print(EXECUTED_PATH[0])
50 G = nx.DiGraph()
51 G.add_edge("Start", c1)
52 G.add_edge(c1, "Positive Path")
53 G.add_edge(c1, c2)
54 G.add_edge("Positive Path", "End")
55 G.add_edge(c2, "Negative Path")
56 G.add_edge(c2, "Zero Path")
```



```
57 G.add_edge("Negative Path", "End")
58 G.add_edge("Zero Path", "End")
59 pos = {
60     "Start": (0, 5),
61     c1: (0, 4),
62     "Positive Path": (-2, 3),
63     c2: (2, 3),
64     "Negative Path": (1, 2),
65     "Zero Path": (3, 2),
66     "End": (0, 1)
67 }
68 edge_labels = {
69     (c1, "Positive Path"): "TRUE",
70     (c1, c2): "FALSE",
71     (c2, "Negative Path"): "TRUE",
72     (c2, "Zero Path"): "FALSE"
73 }
74 highlight_edges = []
75 highlight_nodes = []
76 if EXECUTED_PATH:
77     branch = EXECUTED_PATH[0]
78     if branch == "Positive":
79         highlight_edges = [
80             ("Start", c1),
81             (c1, "Positive Path"),
82             ("Positive Path", "End")
83         ]
84         highlight_nodes = [
85             "Start",
86             c1,
87             "Positive Path",
88             "End"
89         ]
90     elif branch == "Negative":
91         highlight_edges = [
92             ("Start", c1),
93             (c1, c2),
94             (c2, "Negative Path"),
95             ("Negative Path", "End")
96         ]
```

```
97         highlight_nodes = [  
98             "Start",  
99             c1,  
100             c2,  
101             "Negative Path",  
102             "End"  
103         ]  
104     elif branch == "Zero":  
105         highlight_edges = [  
106             ("Start", c1),  
107             (c1, c2),  
108             (c2, "Zero Path"),  
109             ("Zero Path", "End")  
110         ]  
111         highlight_nodes = [  
112             "Start",  
113             c1,  
114             c2,  
115             "Zero Path",  
116             "End"  
117         ]  
118     node_colors = []  
119     for node in G.nodes():  
120         if node in highlight_nodes:  
121             node_colors.append("red")  
122         else:  
123             node_colors.append("skyblue")  
124     plt.figure(figsize=(12, 8))  
125     nx.draw_networkx_nodes(  
126         G,  
127         pos,  
128         node_color=node_colors,  
129         node_size=3500  
130     )  
131     nx.draw_networkx_labels(  
132         G,  
133         pos,  
134         font_size=9,  
135         font_color="black"  
136     )
```

```
137 nx.draw_networkx_edges(  
138     G,  
139     pos,  
140     edge_color="black",  
141     arrows=True  
142 )  
143 nx.draw_networkx_edge_labels(  
144     G,  
145     pos,  
146     edge_labels=edge_labels,  
147     font_size=10  
148 )  
149 if highlight_edges:  
150     nx.draw_networkx_edges(  
151         G,  
152         pos,  
153         edgelist=highlight_edges,  
154         edge_color="red",  
155         width=4,  
156         arrows=True  
157     )  
158 plt.title("Control Flow Graph (Executed Path Highlighted)")  
159 plt.axis("off")  
160 plt.show()
```

### 7.1.2 Sample Output

PATH TESTING ANALYSIS :-

Decision Nodes = 2

Cyclomatic Complexity = 3

Independent Paths:

Path 1

Start -> IF(b > 0) TRUE -> End

Path 2

Start -> IF(b > 0) FALSE -> IF(b < 0) TRUE -> End

Path 3

Start -> IF(b > 0) FALSE -> IF(b < 0) FALSE -> End

Executing Program...

Enter a number: 5

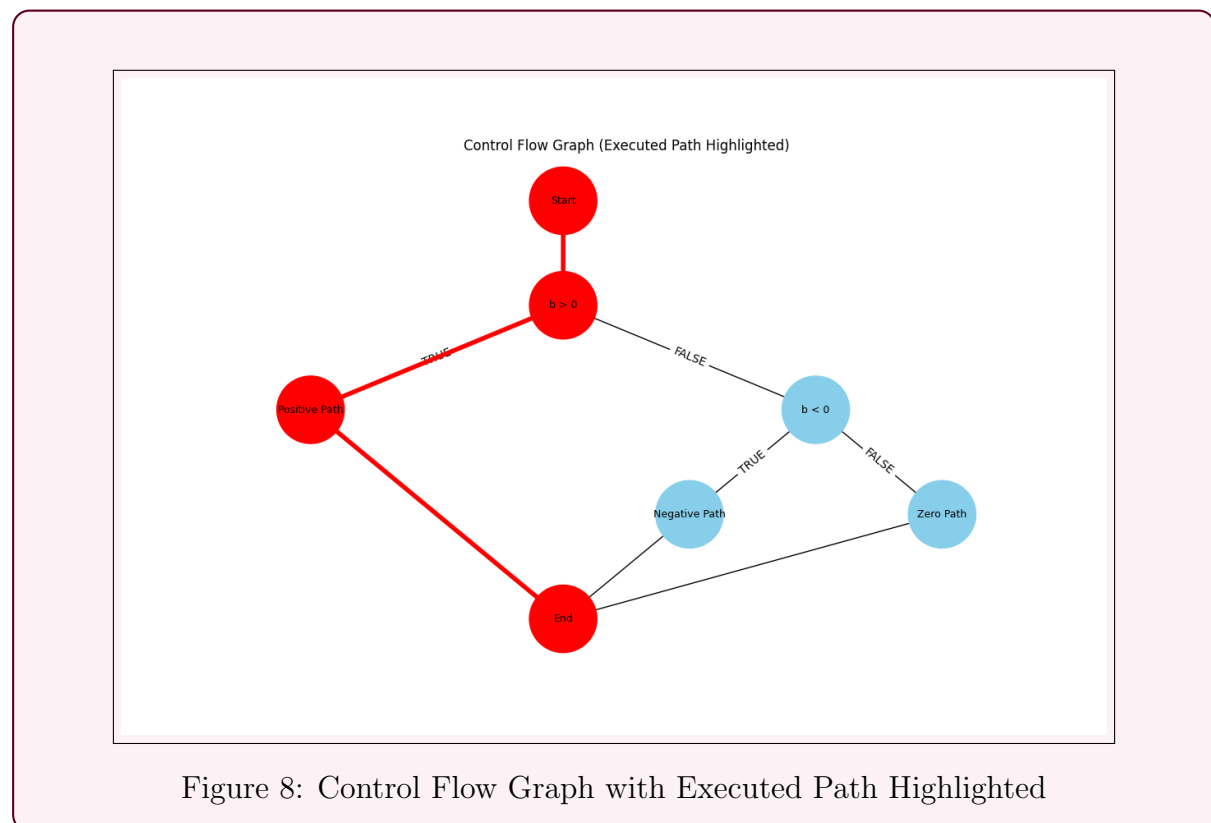
Positive

Program Finished

Executed Branch:

Positive

### 7.1.3 Control Flow Graph



## Result

The Path Testing program successfully analyzed the source code, identified all decision nodes, calculated the Cyclomatic Complexity, and generated the independent execution paths. During execution, the program correctly identified the executed branch and highlighted the corresponding path in the Control Flow Graph. This confirms that the program can effectively support white-box testing by ensuring adequate path coverage.

## 7.2 Boundary Value Analysis

### Objective

To identify the boundary values of input conditions in a program and generate valid and invalid boundary test cases using the Boundary Value Analysis (BVA) technique.

### Input Program (input.py)

```
1 age = int(input())
2 if age < 18 or age > 60:
3     print("Invalid")
4 else:
5     print("Valid")
```

#### 7.2.1 Python Program

```
1 import ast
2 with open("boundary_value_analysis/input.py", "r") as f:
3     code = f.read()
4 tree = ast.parse(code)
5 print("BOUNDARY VALUE ANALYSIS")
6 for node in ast.walk(tree):
7     if not isinstance(node, ast.If):
8         continue
9     print("\nCondition Found:")
10    print(ast.unparse(node.test))
11    # ----- RANGE CONDITION -----
12    if isinstance(node.test, ast.BoolOp):
13        values = node.test.values
14        if len(values) == 2:
15            c1 = values[0]
16            c2 = values[1]
17            if (
18                isinstance(c1, ast.Compare)
19                and isinstance(c2, ast.Compare)
20            ):
21                var = ast.unparse(c1.left)
22                try:
23                    v1 = int(ast.unparse(c1.comparators[0]))
```

```

24         v2 = int(ast.unparse(c2.comparators[0]))
25         op1 = c1.ops[0]
26         op2 = c2.ops[0]
27         if isinstance(op1, ast.Lt) and isinstance(op2, ast.Gt):
28             lower = v1
29             upper = v2
30         elif isinstance(op1, ast.Gt) and isinstance(op2,
31             ↪ ast.Lt):
32             lower = v2
33             upper = v1
34         else:
35             continue
36         print("\nVALID RANGE")
37         print(f"{lower} <= {var} <= {upper}")
38         print("\nBOUNDARY VALUES")
39         print(f"Lower Boundary : {lower}")
40         print(f"Upper Boundary : {upper}")
41         print("\nVALID BOUNDARY VALUES")
42         print(f"{lower}")
43         print(f"{lower + 1}")
44         print(f"{upper - 1}")
45         print(f"{upper}")
46         print("\nINVALID BOUNDARY VALUES")
47         print(f"{lower - 1}")
48         print(f"{upper + 1}")
49         print("\nBOUNDARY TEST CASES")
50         print(f"TC1 = {lower - 1} (Invalid)")
51         print(f"TC2 = {lower} (Valid)")
52         print(f"TC3 = {lower + 1} (Valid)")
53         print(f"TC4 = {upper - 1} (Valid)")
54         print(f"TC5 = {upper} (Valid)")
55         print(f"TC6 = {upper + 1} (Invalid)")
56     except:
57         pass
58 # ----- SINGLE CONDITION -----
59 elif isinstance(node.test, ast.Compare):
60     var = ast.unparse(node.test.left)
61     op = node.test.ops[0]
62     try:
63         value = int(ast.unparse(node.test.comparators[0]))

```

```
63     print("\nBOUNDARY")
64     print(value)
65     print("\nVALID VALUES")
66     if isinstance(op, ast.Gt):
67         print(value + 1)
68     elif isinstance(op, ast.GtE):
69         print(value)
70         print(value + 1)
71     elif isinstance(op, ast.Lt):
72         print(value - 1)
73     elif isinstance(op, ast.LtE):
74         print(value)
75         print(value - 1)
76     elif isinstance(op, ast.Eq):
77         print(value)
78     print("\nINVALID VALUES")
79     if isinstance(op, ast.Gt):
80         print(value)
81         print(value - 1)
82     elif isinstance(op, ast.GtE):
83         print(value - 1)
84     elif isinstance(op, ast.Lt):
85         print(value)
86         print(value + 1)
87     elif isinstance(op, ast.LtE):
88         print(value + 1)
89     elif isinstance(op, ast.Eq):
90         print(value - 1)
91         print(value + 1)
92     print("\nBOUNDARY TEST CASES")
93     print(f"{value-1}")
94     print(f"{value}")
95     print(f"{value+1}")
96 except:
97     pass
```

### 7.2.2 Sample Output

#### BOUNDARY VALUE ANALYSIS

Condition Found:

age < 18 or age > 60

#### VALID RANGE

18 <= age <= 60

#### BOUNDARY VALUES

Lower Boundary : 18

Upper Boundary : 60

#### VALID BOUNDARY VALUES

18

19

59

60

#### INVALID BOUNDARY VALUES

17

61

#### BOUNDARY TEST CASES

TC1 = 17 (Invalid)

TC2 = 18 (Valid)

TC3 = 19 (Valid)

TC4 = 59 (Valid)

TC5 = 60 (Valid)

TC6 = 61 (Invalid)

### Boundary Test Cases



| Test Case | Input Value | Expected Result |
|-----------|-------------|-----------------|
| TC1       | 17          | Invalid         |
| TC2       | 18          | Valid           |
| TC3       | 19          | Valid           |
| TC4       | 59          | Valid           |
| TC5       | 60          | Valid           |
| TC6       | 61          | Invalid         |

## Result

The Boundary Value Analysis program successfully analyzed the input condition, identified the valid input range, determined the lower and upper boundary values, and generated both valid and invalid boundary test cases. The generated test cases effectively verify the behavior of the program at and around its boundary conditions, improving the likelihood of detecting boundary-related errors.

## 7.3 Equivalence Class Partitioning

### Objective

To identify valid and invalid equivalence classes from the input conditions of a program and generate representative test cases using the Equivalence Class Partitioning (ECP) technique.

### Input Program (input.py)

```
1 age = int(input("Enter age: "))
2 if age > 20 or age < 10:
3     print("allowed")
4 else:
5     print("not allowed")
```

#### 7.3.1 Python Program

```
1 import ast
2 with open("equivalent_class_partitioning/input.py", "r") as f:
3     code = f.read()
4 tree = ast.parse(code)
5 for node in ast.walk(tree):
6     if not isinstance(node, ast.If):
7         continue
8     condition = ast.unparse(node.test)
9     print("\nCondition Found:")
10    print(condition)
11    if isinstance(node.test, ast.BoolOp):
12        values = node.test.values
13        if len(values) == 2:
14            c1 = values[0]
15            c2 = values[1]
16            if (
17                isinstance(c1, ast.Compare)
18                and isinstance(c2, ast.Compare)
19            ):
20                var1 = ast.unparse(c1.left)
21                var2 = ast.unparse(c2.left)
22                if var1 == var2:
```

```
23         op1 = c1.ops[0]
24         op2 = c2.ops[0]
25     try:
26         val1 = int(ast.unparse(c1.comparators[0]))
27         val2 = int(ast.unparse(c2.comparators[0]))
28         # age > 18 OR age < 0
29         if isinstance(op1, ast.Gt) and isinstance(op2,
30             ↪ ast.Lt):
31             print("\nVALID EQUIVALENCE CLASS")
32             print(
33                 f"EC1 : {val2} <= {var1} <= {val1}"
34             )
35             print("\nINVALID EQUIVALENCE CLASSES")
36             print(
37                 f"EC2 : {var1} < {val2}"
38             )
39             print(
40                 f"EC3 : {var1} > {val1}"
41             )
42             print("\nREPRESENTATIVE TEST CASES")
43             print(
44                 f"TC1 = {(val1+val2)//2} (Valid)"
45             )
46             print(
47                 f"TC2 = {val2-1} (Invalid)"
48             )
49             print(
50                 f"TC3 = {val1+1} (Invalid)"
51             )
52         # marks < 0 OR marks > 100
53         elif isinstance(op1, ast.Lt) and isinstance(op2,
54             ↪ ast.Gt):
55             print("\nVALID EQUIVALENCE CLASS")
56             print(
57                 f"EC1 : {val1} <= {var1} <= {val2}"
58             )
59             print("\nINVALID EQUIVALENCE CLASSES")
60             print(
61                 f"EC2 : {var1} < {val1}"
62             )
```

```
61         print(  
62             f"EC3 : {var1} > {val2}"  
63         )  
64         print("\nREPRESENTATIVE TEST CASES")  
65         print(  
66             f"TC1 = {(val1+val2)//2} (Valid)"  
67         )  
68         print(  
69             f"TC2 = {val1-1} (Invalid)"  
70         )  
71         print(  
72             f"TC3 = {val2+1} (Invalid)"  
73         )  
74     except:  
75         pass  
76 elif isinstance(node.test, ast.Compare):  
77     left = ast.unparse(node.test.left)  
78     op = node.test.ops[0]  
79     right = ast.unparse(node.test.comparators[0])  
80     try:  
81         r = int(right)  
82         print("\nINVALID EQUIVALENCE CLASS")  
83         if isinstance(op, ast.Gt):  
84             print(f"EC1 : {left} > {r}")  
85             print("\nINVALID EQUIVALENCE CLASS")  
86             print(f"EC2 : {left} <= {r}")  
87             print("\nREPRESENTATIVE TEST CASES")  
88             print(f"TC1 = {r+1} (Valid)")  
89             print(f"TC2 = {r} (Invalid)")  
90         elif isinstance(op, ast.Lt):  
91             print(f"EC1 : {left} < {r}")  
92             print("\nINVALID EQUIVALENCE CLASS")  
93             print(f"EC2 : {left} >= {r}")  
94             print("\nREPRESENTATIVE TEST CASES")  
95             print(f"TC1 = {r-1} (Valid)")  
96             print(f"TC2 = {r} (Invalid)")  
97         elif isinstance(op, ast.GtE):  
98             print(f"EC1 : {left} >= {r}")  
99             print("\nINVALID EQUIVALENCE CLASS")  
100            print(f"EC2 : {left} < {r}")
```

```
101         print("\nREPRESENTATIVE TEST CASES")
102         print(f"TC1 = {r} (Valid)")
103         print(f"TC2 = {r-1} (Invalid)")
104     elif isinstance(op, ast.LtE):
105         print(f"EC1 : {left} <= {r}")
106         print("\nINVALID EQUIVALENCE CLASS")
107         print(f"EC2 : {left} > {r}")
108         print("\nREPRESENTATIVE TEST CASES")
109         print(f"TC1 = {r} (Valid)")
110         print(f"TC2 = {r+1} (Invalid)")
111     elif isinstance(op, ast.Eq):
112         print(f"EC1 : {left} == {r}")
113         print("\nINVALID EQUIVALENCE CLASS")
114         print(f"EC2 : {left} != {r}")
115         print("\nREPRESENTATIVE TEST CASES")
116         print(f"TC1 = {r} (Valid)")
117         print(f"TC2 = {r+1} (Invalid)")
118 except:
119     pass
```

### 7.3.2 Sample Output

```
Condition Found:
age > 20 or age < 10

VALID EQUIVALENCE CLASS
EC1 : 10 <= age <= 20

INVALID EQUIVALENCE CLASSES
EC2 : age < 10
EC3 : age > 20

REPRESENTATIVE TEST CASES
TC1 = 15 (Valid)
TC2 = 9 (Invalid)
TC3 = 21 (Invalid)
```

## Equivalence Classes

| Class | Description                   |
|-------|-------------------------------|
| EC1   | $10 \leq age \leq 20$ (Valid) |
| EC2   | $age < 10$ (Invalid)          |
| EC3   | $age > 20$ (Invalid)          |

## Representative Test Cases

| Test Case | Input Value | Expected Result |
|-----------|-------------|-----------------|
| TC1       | 15          | Valid           |
| TC2       | 9           | Invalid         |
| TC3       | 21          | Invalid         |

## Result

The Equivalence Class Partitioning program successfully analyzed the input condition and divided the input domain into valid and invalid equivalence classes. Representative test cases were generated for each class, enabling efficient testing with a reduced number of test cases while maintaining adequate coverage of the input domain.

## 7.4 Data Flow Testing

### Objective

To perform Data Flow Testing on a Python program by identifying variable definitions and uses, constructing Definition-Use (DU) chains, and generating the corresponding Control Flow Graph (CFG).

### Input Program (input.py)

```
1  a = 10
2  b = 20
3  c = a + b
4  if c > 25:
5      d = c * 2
6  print(d)
```

#### 7.4.1 Python Program

```
1  import ast
2  import networkx as nx
3  import matplotlib.pyplot as plt
4  from tabulate import tabulate
5  # READ SOURCE FILE
6  with open("data_flow_testing/input.py", "r") as f:
7      code = f.read()
8  lines = code.splitlines()
9  print("DATA FLOW TESTING")
10 print("\nSOURCE PROGRAM\n")
11 for i, line in enumerate(lines, start=1):
12     print(f"{i:2d} | {line}")
13 # AST ANALYSIS
14 tree = ast.parse(code)
15 definitions = {}
16 uses = {}
17 du_chains = []
18 # VISITOR
19 class DataFlowVisitor(ast.NodeVisitor):
20     def visit_Assign(self, node):
21         # Definitions
```

```

22     for target in node.targets:
23         if isinstance(target, ast.Name):
24             var = target.id
25             definitions.setdefault(var, [])
26             definitions[var].append(node.lineno)
27         # Uses inside assignment expression
28     for child in ast.walk(node.value):
29         if isinstance(child, ast.Name):
30             var = child.id
31             uses.setdefault(var, [])
32             uses[var].append(node.lineno)
33     self.generic_visit(node)
34 def visit_If(self, node):
35     # Predicate Uses (P-Use)
36     for child in ast.walk(node.test):
37         if isinstance(child, ast.Name):
38             var = child.id
39             uses.setdefault(var, [])
40             uses[var].append(node.lineno)
41     self.generic_visit(node)
42 def visit_Name(self, node):
43     if isinstance(node.ctx, ast.Load):
44         parent = getattr(node, "parent", None)
45         if not isinstance(parent, ast.Assign):
46             uses.setdefault(node.id, [])
47             if node.lineno not in uses[node.id]:
48                 uses[node.id].append(node.lineno)
49     self.generic_visit(node)
50 # Parent links
51 for parent in ast.walk(tree):
52     for child in ast.iter_child_nodes(parent):
53         child.parent = parent
54 visitor = DataFlowVisitor()
55 visitor.visit(tree)
56 # BUILD DU TABLE
57 table = []
58 all_vars = sorted(
59     set(definitions.keys()) | set(uses.keys())
60 )
61 for var in all_vars:

```



```

62     def_lines = definitions.get(var, [])
63     use_lines = uses.get(var, [])
64     table.append([
65         var,
66         ", ".join(map(str, def_lines)) if def_lines else "-",
67         ", ".join(map(str, use_lines)) if use_lines else "-"
68     ])
69     print("DEFINITION / USE TABLE")
70     print(
71         tabulate(
72             table,
73             headers=[
74                 "Variable",
75                 "Defined At",
76                 "Used At"
77             ],
78             tablefmt="grid"
79         )
80     )
81     # DU CHAINS
82     print("DU CHAINS")
83     for var in all_vars:
84         defs = definitions.get(var, [])
85         uses_list = uses.get(var, [])
86         if defs and uses_list:
87             latest_def = defs[0]
88             for u in uses_list:
89                 while (
90                     len(defs) > 1 and
91                     defs[1] < u
92                 ):
93                     defs.pop(0)
94                     latest_def = defs[0]
95                 print(
96                     f"{var}: D({latest_def}) -> U({u})"
97                 )
98     # CFG GENERATION
99     G = nx.DiGraph()
100    G.add_node("Start")
101    previous = "Start"

```

```
102 node_num = 1
103 for stmt in tree.body:
104     if isinstance(stmt, ast.Assign):
105         label = f"{node_num}. Assign"
106         G.add_node(label)
107         G.add_edge(previous, label)
108         previous = label
109         node_num += 1
110     elif isinstance(stmt, ast.If):
111         cond = f"{node_num}. IF"
112         G.add_node(cond)
113         G.add_edge(previous, cond)
114         true_node = f"{node_num}. True"
115         G.add_node(true_node)
116         G.add_edge(cond, true_node)
117         merge = f"{node_num}. Merge"
118         G.add_node(merge)
119         G.add_edge(true_node, merge)
120         G.add_edge(cond, merge)
121         previous = merge
122         node_num += 1
123     else:
124         label = f"{node_num}. Statement"
125         G.add_node(label)
126         G.add_edge(previous, label)
127         previous = label
128         node_num += 1
129 G.add_node("End")
130 G.add_edge(previous, "End")
131 # DRAW CFG
132 plt.figure(figsize=(10, 7))
133 pos = nx.spring_layout(
134     G,
135     seed=42
136 )
137 nx.draw(
138     G,
139     pos,
140     with_labels=True,
141     node_size=3000,
```

```

142     arrows=True,
143     font_size=9
144 )
145 plt.title("Control Flow Graph (CFG)")
146 plt.axis("off")
147 plt.show()

```

### 7.4.2 Sample Output

#### DATA FLOW TESTING

##### SOURCE PROGRAM

```

1 | a = 10
2 | b = 20
3 | c = a + b
4 | if c > 25:
5 |     d = c * 2
6 | print(d)

```

##### DEFINITION / USE TABLE

| Variable | Defined At | Used At |
|----------|------------|---------|
| a        | 1          | 3       |
| b        | 2          | 3       |
| c        | 3          | 4, 5    |
| d        | 5          | 6       |
| print    | -          | 6       |

##### DU CHAINS

```

a: D(1) -> U(3)
b: D(2) -> U(3)
c: D(3) -> U(4)
c: D(3) -> U(5)
d: D(5) -> U(6)

```

### Definition-Use Table

| Variable | Defined At | Used At |
|----------|------------|---------|
| a        | 1          | 3       |
| b        | 2          | 3       |
| c        | 3          | 4, 5    |
| d        | 5          | 6       |
| print    | --         | 6       |

### 7.4.3 Control Flow Graph

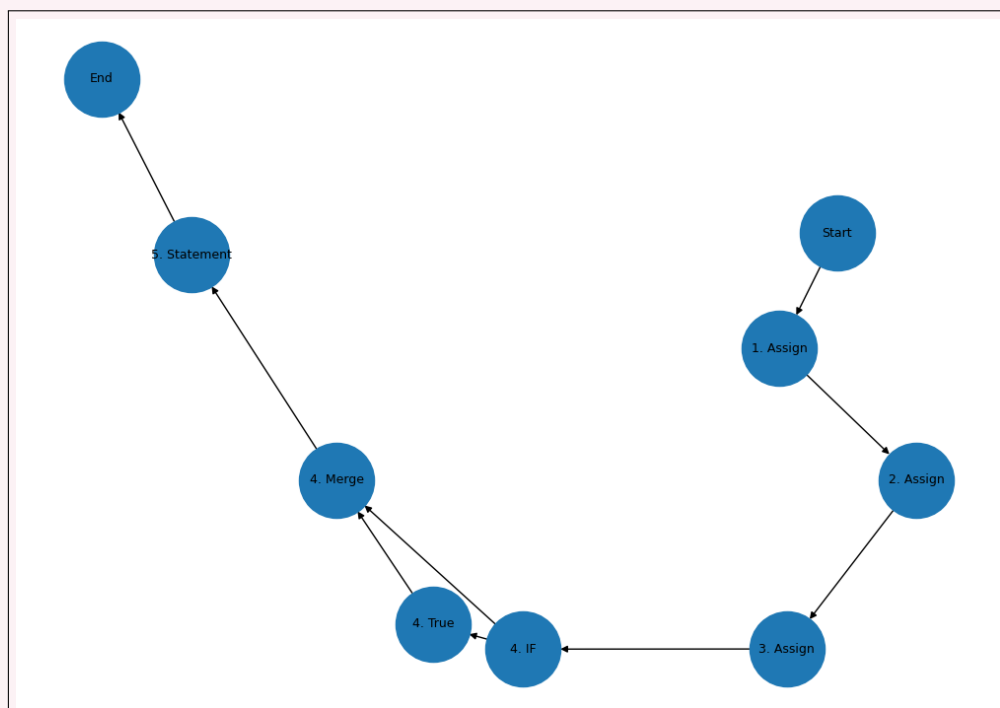


Figure 9: Control Flow Graph generated for Data Flow Testing

### Result

The Data Flow Testing program successfully analyzed the source code and identified all variable definitions and uses. The Definition-Use table and DU chains clearly illustrate the flow of data throughout the program, while the generated Control Flow Graph provides a visual representation of the program's execution paths. This analysis confirms that each variable is properly defined before use, demonstrating the effectiveness of Data Flow Testing in identifying data-related defects.